

Advanced OOP (Object-Oriented Programming)

“System banana seekh lo, script likhna band karo.”

◇ Core Idea (One-Line)

OOP = Blueprint banaao → unlimited reusable machines chalao → system scale karo.

Instead of:

- 1 long messy script ❌ You build:
 - Small **blueprints (Classes)** → many **working objects (Instances)** ☑
-

Real-Life Analogy (IMPORTANT)

Think like a **Factory Owner**, not a labourer.

- **Class** → Factory blueprint
 - **Object** → Actual machine made from blueprint
 - **Method** → What the machine can do
 - **Attributes** → Machine ka data/state
-

1 Class vs Object (Instance)

Simple Hinglish

- **Class** = Design / Blueprint
- **Object** = Real thing created from that design

Example

```
Car (Class)
|
|- car1 (Object)
|- car2 (Object)
|- car3 (Object)
```

Your System Example

```
Scraper (Class)
|
|- cloudflare_scraper (Object)
|- medium_scraper (Object)
|- blog_scraper (Object)
```

Why it matters for you? You will build:

- User class
- Question class
- Test class

Without rewriting logic again and again.

2 `__init__` – Constructor (Birth Certificate)

Meaning

`__init__` runs **automatically** when object is created.

Real Life

- Jab banda paida hota hai → naam, age assign hoti hai
- Jab object paida hota hai → data assign hota hai

Example

```
class User:
    def __init__(self, name, email):
        self.name = name
        self.email = email
```

```
u1 = User("Arun", "arun@gmail.com")
```

 Object **born**, data set.

Rule: If no `__init__` → object has no initial state → useless.

3 `self` – “Ye wala object”

 Simple Truth

`self` = **current object ka reference**

 Analogy

You say:

- “**Main** kaam kar raha hoon”

Python says:

- “**self** kaam kar raha hoon”

✗ Common Confusion

`self` is NOT keyword It's just a convention (but mandatory practically).

 Example

```
class User:
    def greet(self):
        print("Hello", self.name)
```

Each object has its **own self.name**.

4 Inheritance – Parent → Child (Reuse Power)

 Idea

Ek **base class** banao Usse **specialized child classes** nikalo

 Real Life

- Parent → General rules
- Child → Extra abilities

 Your System Example

```
BaseScraper
├── BlogScraper
├── NewsScraper
└── MCQScraper
```

🔑 Example

```
class BaseScraper:
    def fetch(self):
        print("Fetching page")

class BlogScraper(BaseScraper):
    def parse(self):
        print("Parsing blog")
```

Why this is GOLD for you?

- One fetch logic
- Multiple site-specific parsers
- Clean, scalable scraping system

5 Encapsulation – Data ko lock karo

🧠 Meaning

Internal data ko **direct access se bachana**

Real Life

- ATM machine
- You don't see backend logic
- Sirf buttons milte hain

🔑 Python Example

```
class User:
    def __init__(self, password):
        self.__password = password
```

- `__password` = **private**
- Bahar se access ✗
- Class ke andar allowed ✓

Why YOU need this

- Passwords
- Tokens
- API keys
- Exam answers

Encapsulation = **security** + **sanity**

6 Polymorphism – Same action, different behavior

Meaning

Same method name Different implementation

Real Life

- Dog speaks → Bark
- Cat speaks → Meow Same word: *speak()*

Example

```
class Scraper:
    def parse(self):
        print("Generic parsing")

class BlogScraper(Scraper):
    def parse(self):
        print("Parsing blog HTML")

class NewsScraper(Scraper):
    def parse(self):
        print("Parsing news HTML")
```

Power Move

```
scrapers = [BlogScraper(), NewsScraper()]

for s in scrapers:
    s.parse()
```

→ Same loop → Different behavior → ZERO if-else hell

SYSTEM DESIGN MENTAL MODEL (IMPORTANT)

✗ Bad Thinking

"Script chal raha hai bas"

☑ Builder Thinking

"Is this reusable? extensible? testable?"

How This Fits Your NEETPrepGPT System

Entity	Why Class?
User	state (email, role, score)
Question	data + validation
Test	logic + evaluation
Scraper	reusable extraction engine

Final Rulebook (Print this in head)

- **Class = system component**
 - **Object = running instance**
 - **Inheritance = reuse**
 - **Encapsulation = safety**
 - **Polymorphism = flexibility**
 - **No long scripts after this ✗**
-